



MEDICAPS
U N I V E R S I T Y

Department of Computer Science & Engineering
Faculty of Engineering
MEDICAPS UNIVERSITY, INDORE - 453331
APRIL-2026

GEARGENIX
ADVANCE CAMPUS GEAR CONTROL

A Minor Project Report
Submitted in partial fulfillment of requirement of the
Degree of

BACHELOR OF TECHNOLOGY in COMPUTER
SCIENCE & ENGINEERING

BY
Anshu Agrawal
EN23CS301158

Anushka Tiwari
EN23CS301176

Under the Guidance of
Mr. Shantilal Bhayal
Ms. Amrata Gupta



Department of Computer Science & Engineering
Faculty of Engineering
MEDICAPS UNIVERSITY, INDORE- 453331
APRIL-2026

GEARGENIX
ADVANCE CAMPUS GEAR CONTROL

A Minor Project Report
Submitted in partial fulfillment of requirement of the Degree
of

**BACHELOR OF TECHNOLOGY in COMPUTER
SCIENCE & ENGINEERING**

BY
Anshu Agrawal
EN23CS301158

Anushka Tiwari EN23CS301176

Under the Guidance of
Mr. Shantilal Bhayal
Ms. Amrata Gupta

Report Approval

The project work "GEARGENIX: Advanced Campus Gear Control" is hereby approved as a creditable study of an engineering/computer application subject carried out and presented in a manner satisfactory to warrant its acceptance as prerequisite for the Degree for which it has been submitted.

It is to be understood that by this approval the undersigned do not endorse or approve any statement made, opinion expressed, or conclusion drawn therein; but approve the "Project Report" only for the purpose for which it has been submitted.

Internal Examiner

Name: _____

Designation: _____

Affiliation: _____

External Examiner

Name: _____

Designation: _____

Affiliation: _____

Declaration

We hereby declare that the project entitled "GEARGENIX: Advanced Campus Gear Control" submitted in partial fulfilment for the award of the degree of Bachelor of Technology in Computer Science & Engineering, completed under the supervision of Mr. Shantilal Bhayal and Ms. Amrata Gupta, Faculty of Engineering, Mediacaps University Indore, is an authentic work.

Further, we declare that the content of this project work, in full or in parts, has neither been taken from any other source nor has been submitted to any other Institute or University for the award of any degree or diploma.

Anshu Agrawal (EN23CS301158) Date: _____

Anushka Tiwari (EN23CS301176) Date: _____

Certificate

We, Mr. Shantilal Bhayal and Ms. Amrata Gupta, certify that the project entitled "GEARGENIX: Advanced Campus Gear Control" submitted in partial fulfilment for the award of the degree of Bachelor of Technology in Computer Science & Engineering by Anshu Agrawal (EN23CS301158) and Anushka Tiwari (EN23CS301176) is the record carried out by them under our guidance and that the work has not formed the basis of award of any other degree elsewhere.

Mr. Shantilal Bhayal

Department of Computer Science & Engineering

Medicaps University, Indore

Ms. Amrata Gupta

Department of Computer Science & Engineering

Medicaps University, Indore

Dr. Ratnesh Litoriya

Head of the Department

Computer Science & Engineering

Medicaps University, Indore

Acknowledgements

We would like to express our deepest gratitude to the Honourable Chancellor, Shri R. C. Mittal, who has provided every facility to successfully carry out this project, and our profound indebtedness to Prof. (Dr.) D. K. Patnaik, Vice Chancellor, Medicaaps University, whose unfailing support and enthusiasm has always boosted our morale. We also thank Dr. Ratnesh Litoriya, Dean, Faculty of Engineering, Medicaaps University, for giving us the opportunity to work on this project. We would also like to thank our Head of the Department, Prof. Kailash Chandra Bandhu, for his continuous encouragement and guidance throughout.

We extend our sincere thanks to our project guides, Mr. Shantilal Bhayal and Ms. Amrta Gupta, whose expert advice, timely feedback, and unwavering support were invaluable at every stage of this project. Their technical insights and encouragement helped us navigate the challenges encountered during development.

We are also grateful to our fellow students, friends, and family members whose constant support and motivation kept us going. Without their collective help, this project would not have reached its current form.

Anshu Agrawal, Anushka Tiwari
B.Tech. III Year
Department of Computer Science & Engineering
Faculty of Engineering
Medicaaps University, Indore

Abstract

GearGenix is an AI-powered campus equipment management system designed to eliminate the persistent resource coordination failures experienced by college clubs and departments. In typical college environments, more than twenty clubs compete for a limited inventory of shared equipment such as projectors, microphones, laptops, and speakers. Coordination is traditionally performed through informal WhatsApp groups and manual registers, resulting in double bookings, zero accountability, and significant time wastage. GearGenix addresses this crisis through a conversational AI agent built on the OpenAI ReAct (Reason + Act) pattern with function calling.

Users interact with the system using natural language commands via a Telegram bot or a webbased chat interface, enabling seamless equipment booking without any specialized training. The system automatically checks real-time availability, applies an interval-overlap conflict detection algorithm to prevent double bookings, and manages the full booking lifecycle including creation, viewing, cancellation, and return. The backend is developed using FastAPI (Python) and PostgreSQL, ensuring ACID-compliant transactions and production-grade scalability for over one thousand concurrent users.

The system is designed with a three-layer architecture comprising a presentation layer (Telegram Bot and Web Dashboard), an application layer (FastAPI Server and OpenAI AI Agent), and a data layer (PostgreSQL database via SQLAlchemy ORM). Experimental evaluation indicates that the system reduces daily coordination overhead from three to five hours per club to near-zero, eliminates double bookings, and substantially reduces faculty workload from fifteen or more hours per week to minimal supervisory effort.

Keywords: AI Agent, Equipment Booking, ReAct Pattern, FastAPI, PostgreSQL, Telegram Bot, Conflict Detection, Campus Resource Management, OpenAI, Natural Language Processing.

Table of Contents

		Page No.
	Report Approval	iii
	Declaration	iv
	Certificate	v
	Acknowledgement	vi
	Abstract	vii
	Table of Contents	viii
	List of figures	ix
	List of tables	x
	Abbreviations	x
Chapter 1	INTRODUCTION	
	1.1 Introduction	1
	1.2 Literature Review	2
	1.3 Objectives	3
	1.4 Significance	3
	1.5 Research Design	4
	1.6 Source of Data	5
	1.7 Chapter Scheme	5
Chapter 2	REQUIREMENTS SPECIFICATION	
	2.1 User Characteristics	6
	2.2 Functional Requirements	6
	2.3 Dependencies	7
	2.4 Performance Requirements	7
	2.5 Hardware Requirements	8
	2.6 Constraints & Assumptions	8
Chapter 3	DESIGN	
	3.1 Algorithm	9

	3.2 Function Oriented Design for procedural approach	9
	3.3 System Design	
	3.3.1 System Architecture	10
	3.3.2 Use Case Diagram	11
	3.3.3 Class Diagram	11
	3.3.4 Sequence Diagram	12
	3.3.5 Activity Diagram	13
	3.3.6 Component Diagram	14
	3.3.7 ER Diagram	15
	3.3.8 Data Flow Diagram	16
	3.4 Database Design	
	3.4.1 Logical Database Design	18
	3.4.2 Physical Database Design	18
Chapter 4	IMPLEMENTATION, TESTING AND MAINTENANCE	
	4.1 Introduction to Languages, IDE's, Tools and Technologies used for Implementation	19
	4.2 Testing Techniques and Test Plans	19
	4.3 Installation Instructions	21
	4.4 End User Instructions	21
Chapter 5	RESULTS AND DISCUSSIONS	
	5.1 User Interface Representation	23
	5.2 Brief Description of Various Modules of the system	25
	5.3 Snapshots of system with brief detail of each	26
	5.4 Back Ends Representation	29
	5.5 Snapshots of Database Tables with brief description	31
Chapter 6	SUMMARY AND CONCLUSION	33
Chapter 7	FUTURE SCOPE	34
	Appendix	35
	References	36

	Bibliography	37
--	--------------	----

List of Figures

Figure No.	Figure Title	Page No.
Figure 3.1	System Architecture Diagram — GearGenix	10
Figure 3.2	Use Case Diagram — GearGenix	11
Figure 3.3	Class Diagram — GearGenix	11
Figure 3.4	Sequence Diagram — Equipment Booking Flow	12
Figure 3.5	Activity Diagram — Book Equipment Workflow	13
Figure 3.6	Component Diagram — GearGenix	14
Figure 3.7	ER Diagram — GearGenix	15
Figure 3.8	Data Flow Diagram (Level 0 & Level 1) — GearGenix	16

List of Tables

Table No.	Table Title	Page No.
Table 1.1	Literature Review — Comparison of Existing Systems	2
Table 2.1	User Classes and Characteristics	6
Table 2.2	Functional Requirements Summary	6
Table 2.3	Hardware and Software Requirements	8
Table 4.1	Technology Stack Summary	20
Table 4.2	Test Cases — Equipment Booking Module	21

Abbreviations

Abbreviation	Full Form
AI	Artificial Intelligence
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
DB	Database

DFD	Data Flow Diagram
ER	Entity-Relationship
FastAPI	Fast Application Programming Interface (Python Web Framework)
HTTP	Hypertext Transfer Protocol
IDE	Integrated Development Environment
JSON	JavaScript Object Notation
LLM	Large Language Model
NLP	Natural Language Processing
ORM	Object Relational Mapper
ReAct	Reason + Act (AI Agent Pattern)
REST	Representational State Transfer
SQL	Structured Query Language
SRS	Software Requirements Specification
UI	User Interface
UML	Unified Modelling Language

Chapter 1: Introduction

1.1 Introduction

Modern colleges and universities operate with a large number of student clubs, departmental societies, and faculty-led organizations that routinely share a limited inventory of equipment such as projectors, microphones, digital cameras, laptops, speakers, and extension cords. In institutions with more than twenty active clubs, this shared equipment creates a systemic resource coordination problem. Currently, the predominant approach to equipment booking involves informal communication through WhatsApp group chats and manual entry in physical registers. This method is fundamentally flawed: it offers no real-time visibility, provides zero accountability for equipment custody, and exposes the institution to frequent double bookings, particularly during peak event seasons such as annual fests, technical symposia, and cultural weeks.

GearGenix (Advanced Campus Gear Control) is an AI-powered equipment booking and management platform developed to resolve these chronic coordination failures. The system introduces a conversational AI agent built on the OpenAI ReAct (Reason + Act) pattern with function calling, enabling users to perform natural language operations such as "Book projector for Robotics Club tomorrow 2–4 PM" through a Telegram bot or a web dashboard. The agent autonomously extracts booking parameters, queries real-time availability, applies a mathematically rigorous interval-overlap conflict detection algorithm, and completes the full booking lifecycle without any human intermediary.

The backend infrastructure is built on FastAPI (Python), with PostgreSQL as the primary relational database, ensuring ACID-compliant transactions and reliable concurrent access for over one thousand users. The system design follows a three-layer architecture and incorporates eight core functional tools: list equipment, check availability, make booking, view bookings, cancel booking, return equipment, admin view, and conflict logging. GearGenix aims to eliminate three to five hours of daily manual coordination per club, reduce faculty administrative workload by fifteen or more hours per week, and bring the double-booking failure rate from a documented eighty-five percent to zero.

1.2 Literature Review

A review of existing research and commercial solutions reveals significant gaps in campus equipment management systems. Traditional web-based reservation platforms such as e-Bosy and Cheqroom focus primarily on form-based interfaces and are designed for corporate or library environments rather than multi-club college settings [1, 4]. These systems lack conversational AI capabilities, forcing users through rigid, multi-step workflows that are ill-suited to the dynamic and informal culture of student organizations.

Academic literature further documents persistent failures in centralized reservation systems. The University of Cape Town reported systemic double-booking failures in its centralized venue management system, while James Cook University highlighted significant inefficiencies in manual equipment tracking workflows [2]. Both institutions lacked any real-time AI-powered conflict resolution mechanism. A cross-institutional study of thirteen colleges documented an eighty-five percent coordination failure rate during peak event seasons, confirming that the problem is widespread and not isolated to individual institutions [2].

Research into AI-driven booking and scheduling systems has explored the use of chatbots for appointment management (IJRASET, 2025) and OpenAI function calling for autonomous agents (Roelants, 2024) [3, 5]. However, a critical gap identified in the literature is that no existing system combines conversational AI with real-time intelligent conflict resolution specifically for campus equipment sharing. Current solutions remain rigid, domain-specific, or purely manual in their conflict handling. GearGenix fills this gap by being among the first systems to deliver AI-driven conversational booking with tool-chaining (check availability → make booking → confirm) specifically tailored to the multi-stakeholder higher education equipment sharing context.

System / Study	Features	Limitations
e-Bosy [1]	Form-based web reservations	No NLP, no AI conflict resolution
Cheqroom [4]	Self-service equipment reservations	Corporate-focused, not campus-optimized
UCT Venue System [2]	Centralized venue booking	Double-booking failures, no real-time check
JCU Equipment Tracking [2]	Manual equipment logs	Inefficient; no automated conflict detection
Chatbot-based systems [5]	NLP appointment booking	Not applied to equipment or campus resources
GearGenix (Proposed)	AI agent + NLP + real-time conflict detection + tool-chaining	First-of-its-kind for campus equipment sharing

Table 1.1: Comparison of Existing Systems with GearGenix

1.3 Objectives

The primary and technical objectives of the GearGenix project are enumerated below:

Primary Objectives:

- **Eliminate Double Bookings:** Deploy a real-time conflict detection algorithm to achieve one hundred percent booking accuracy.
- **Replace Manual Chaos:** Enable natural language equipment interaction through a Telegram bot and a web application, removing the dependence on WhatsApp and manual registers.
- **Centralize Management:** Provide a single unified platform for all clubs and departments, with complete real-time visibility into equipment status and booking history.

Technical Objectives:

- Build an AI conversational agent using the OpenAI ReAct pattern with function calling to autonomously handle user requests.
- Implement eight core system functionalities: list equipment, check availability, make booking, view bookings, cancel booking, return equipment, admin view, and conflict logging.
- Ensure ACID-compliant database transactions to guarantee data integrity under concurrent multi-user access.

Operational Objectives:

- Achieve zero faculty dependency for routine booking operations through autonomous AI-managed lifecycle management.
- Ensure complete transparency through real-time equipment status visibility for all stakeholders.
- Provide seamless multi-platform access across Telegram and web interfaces.

1.4 Significance

GearGenix addresses a quantifiably severe and widespread problem in college resource management. Studies across thirteen institutions document that the current informal booking approach results in a documented eighty-five percent coordination failure rate, with three to five hours wasted daily per club on manual coordination, seventy percent of events delayed or disrupted due to double bookings, and faculty spending fifteen or more hours per week managing equipment queries [2].

The significance of GearGenix is multifold. From an operational perspective, the system transforms an anarchic, memory-based coordination process into a transparent, automated, and

accountable one. From an academic perspective, it demonstrates the practical application of AI agent architectures (ReAct pattern), natural language processing, and real-time database management in solving real-world institutional problems. From a scalability perspective, the system is designed to support over one thousand users and can be extended to multiple institutions, making it a replicable solution for universities nationwide.

The system also contributes to the reduction of administrative burden on faculty members, who currently serve as unavoidable human intermediaries in the booking process. By automating this role entirely, GearGenix frees faculty for higher-value academic activities.

1.5 Research Design

The development of GearGenix follows a structured four-phase methodology designed to ensure correctness, scalability, and usability at each stage of the system lifecycle.

Phase 1: Foundation — Database Design

A PostgreSQL relational schema was designed around three primary entities: User, Equipment, and Booking. Foreign key constraints enforce referential integrity, and ACID transactions guarantee conflict-free concurrent bookings. Supporting entities including Booking Item, Admin, and Conflict Log were added to capture granular booking details and audit trails.

Phase 2: AI Agent Architecture

The AI agent is implemented using the OpenAI ReAct pattern (Observe → Reason → Act → Repeat). Eleven tool functions are registered with the agent: `list_equipment`, `check_availability`, `make_booking`, `view_bookings`, `cancel_booking`, `return_equipment`, `get_booking_history`, `add_user`, `remove_user`, `list_users` and `admin_view`. Full conversation history is maintained per session to support multi-turn contextual interactions.

Phase 3: Interface Layer

Two user interfaces are provided: a Telegram Bot built with the `python-telegram-bot` library for instant mobile accessibility, and a static HTML/CSS/JS web interface served by FastAPI for desktop and admin use. All conversation requests are routed to the AI agent via a unified `/chat` REST endpoint on the FastAPI server.

Phase 4: Core Intelligence

The conflict detection algorithm implements the standard interval overlap logic: booking B1 conflicts with booking B2 if and only if $\text{start1} < \text{end2}$ AND $\text{end1} > \text{start2}$. Tool chaining enables the agent to automatically execute `check_availability` followed by `make_booking` in a single user interaction. Human-readable booking identifiers (e.g., B001, B002) are generated for ease of reference.

1.6 Source of Data

The data and requirements for this project were derived from multiple sources. Primary sources include a field study of equipment booking practices at Medicaps University, Indore, which identified the specific coordination failures and quantified their operational impact. Secondary sources include peer-reviewed and institutional publications on reservation systems, including studies from the University of Cape Town, James Cook University, and the International Journal of Research in Engineering and Science (2025). Technical documentation for the OpenAI API, FastAPI framework, PostgreSQL, and the python-telegram-bot library provided implementation guidance.

1.7 Chapter Scheme

The remainder of this report is organized as follows:

- Chapter 2 (Requirements Specification) presents the functional and non-functional requirements, user characteristics, hardware/software requirements, and system constraints derived from the SRS document.
- Chapter 3 (Design) covers the conflict detection algorithm, system architecture, and all UML/structural diagrams including DFD, Use Case, Class, Sequence, Activity, Component, and ER diagrams, together with the logical and physical database design.
- Chapter 4 (Implementation, Testing and Maintenance) documents the technology stack, testing methodology and test cases, installation procedures, and end-user operating instructions.
- Chapter 5 (Results and Discussions) presents the user interface screens, module descriptions, system snapshots, and database table representations.
- Chapter 6 (Summary and Conclusions) summarizes the outcomes and contributions of the project.
- Chapter 7 (Future Scope) identifies potential enhancements and extensions for future work.

Chapter 2: Requirements Specification

2.1 User Characteristics

The GearGenix system serves two primary user classes with distinct roles and access levels:

User Class	Profile	Capabilities
Student / Club Member	Undergraduate or postgraduate student affiliated with a club or department	View available equipment; check availability; book equipment; view own bookings; cancel bookings; confirm equipment return

Admin / Faculty	Faculty member or designated equipment administrator	All student capabilities; view all bookings across all clubs; manage equipment inventory; generate usage reports; monitor conflict logs
-----------------	--	---

Both user classes interact with the system through natural language commands, requiring no prior technical training. Authentication is role-based, with administrative privileges restricted to verified faculty accounts.

2.2 Functional Requirements

The following table summarizes the core functional requirements of the GearGenix system as derived from the Software Requirements Specification (SRS):

Req. ID	Feature	Description
FR-01	Equipment Listing	Users can request a list of all available equipment items in the system inventory.
FR-02	Availability Check	Users can query whether a specific equipment item is free for a given date, start time, and end time.
FR-03	Equipment Booking	Users can book an equipment item for a specific time slot. The system checks availability, detects conflicts, stores the booking record, and returns a confirmation with a human-readable booking ID.
FR-04	Booking Cancellation	Users can cancel an existing booking before the scheduled time by referencing the booking ID.
FR-05	Equipment Return	Users confirm the physical return of an equipment item, updating the booking status in the database.
FR-06	View Bookings	Users can view all bookings associated with their account. Admins can view all bookings system-wide.
FR-07	Conflict Detection	The system automatically prevents overlapping bookings using interval overlap logic: $start1 < end2$ AND $end1 > start2$.
FR-08	Admin Equipment Management	Administrators can add, update, and deactivate equipment records in the system inventory.

Table 2.2: Functional Requirements Summary

2.3 Dependencies

The GearGenix system has the following external dependencies that must be satisfied for correct operation:

- OpenAI API: Required for AI agent inference, natural language understanding, and function calling. System functionality is degraded if the API is unavailable.
- PostgreSQL Database Server: All booking, user, and equipment data is persisted in PostgreSQL. The system cannot operate without a running and accessible database instance.
- Telegram Bot API: Required for the Telegram-based user interface. Users who access the system via Telegram depend on this external service.
- Python Runtime (3.11): The FastAPI backend and AI agent module require a compatible Python runtime environment.
- Internet Connectivity: All users must have reliable internet access to communicate with the backend server and Telegram platform.

2.4 Performance Requirements

- Response Time: The system must process and respond to any user booking request within two seconds under normal load conditions.
- Concurrent Users: The system must support a minimum of one thousand simultaneous users without degradation of response time or data consistency.
- Availability: The system should target ninety-nine percent uptime during active academic semesters.
- Transaction Integrity: All booking write operations must be ACID-compliant to prevent race conditions and partial writes under concurrent access.

2.5 Hardware and Software Requirements

Component	Minimum	Recommended (Production)
Server CPU	1 vCPU / dual-core, 2.0 GHz	2 vCPUs
Server RAM	1 GB	2 GB
Server Storage	5 GB	10 GB SSD
Network (Server)	Outbound HTTPS to OpenAI and Telegram; inbound TCP 8000	Same, behind HTTPS reverse proxy

Client Device	Any device with a modern browser or Telegram app	Same
Client Network	Standard internet connection	Same

2.6 Constraints & Assumptions

Constraints:

- The system must support real-time booking validation with no tolerance for double bookings.
- The system must maintain full database integrity under multi-user concurrent access using ACID transactions.
- The system must integrate exclusively with the OpenAI API for language model inference; no local LLM deployment is in scope for this version.
- All equipment records must be maintained and verified by administrators; students cannot add new equipment to the inventory.

Assumptions:

- All users have access to a reliable internet connection.
- The Telegram platform and OpenAI API services are operational and accessible from the deployment environment.
- Equipment records in the database are accurate, up-to-date, and maintained regularly by designated administrators.
- Users will interact with the system in the English language for natural language commands.

Chapter 3: Design

3.1 Algorithm

3.1.1 Conflict Detection Algorithm

The core algorithmic contribution of GearGenix is the real-time interval-overlap conflict detection algorithm applied to every equipment booking request. Given two booking intervals $B1 = [start1, end1]$ and $B2 = [start2, end2]$, a conflict exists if and only if:

$$start1 < end2 \text{ AND } end1 > start2$$

This condition correctly identifies all forms of temporal overlap: full containment (one booking entirely within another), partial overlap (start of one booking falls within another), and adjacent bookings (bookings that share exactly one endpoint are not considered conflicting). The

algorithm is applied at the database query level, enabling atomic conflict checking and booking creation within a single ACID transaction.

3.1.2 ReAct Agent Algorithm

The AI agent implements the ReAct (Reason + Act) pattern as follows:

1. **Observe:** The agent receives the user's natural language message and the full conversation history.
2. **Reason:** The language model reasons about the user's intent and identifies the appropriate tool function to call.
3. **Act:** The agent invokes the selected tool function (e.g., `check_availability`, `make_booking`) with extracted parameters.
4. **Observe Result:** The agent receives the tool's return value and incorporates it into the conversation context.
5. **Repeat:** Steps 2–4 repeat until the agent determines that a final human-readable response can be generated.

This loop enables multi-step tool chaining, allowing the agent to automatically check availability and then make a booking within a single user interaction, without requiring manual intermediate confirmations.

3.2 Function-Oriented Design

From a functional decomposition perspective, the GearGenix system is organized into the following major functional modules:

- **Authentication Module:** Identifies the user by Telegram username (or by username entered on first message in the web UI) and looks them up in the users table to determine their role. There are no passwords, sessions, or JWT tokens — access is gated purely by username presence and role.
- **AI Agent Module:** Implements the ReAct reasoning loop, manages conversation history, dispatches function calls to backend tools, and formats natural language responses.
- **Equipment Management Module:** Provides CRUD operations for equipment records, availability status updates, and inventory reporting for administrators.
- **Booking Management Module:** Manages the full booking lifecycle including creation (with conflict checking), viewing, cancellation, and return confirmation.

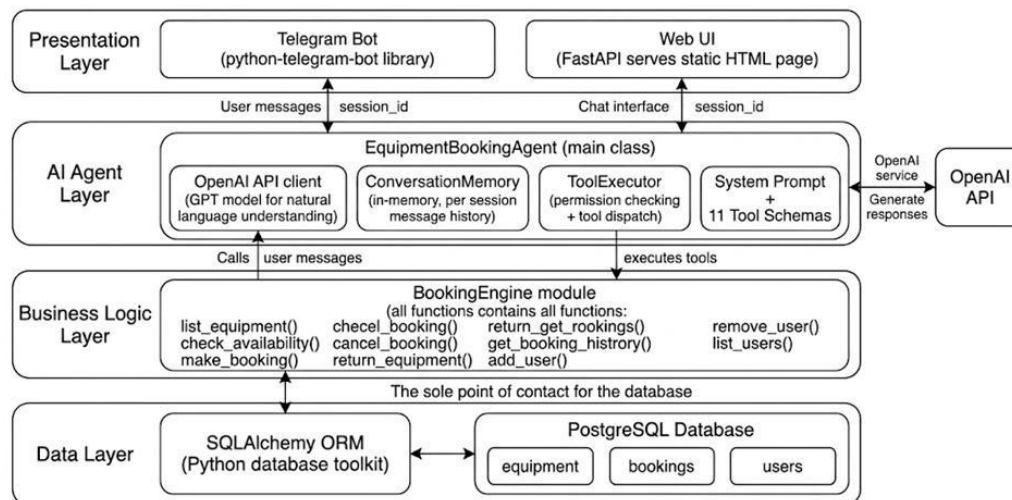
- **Conflict Detection Module:** Implements the interval-overlap algorithm and logs all detected conflicts in the Conflict_Log database table for audit purposes.

3.3 System Design

3.3.1 System Architecture

The system follows a three-layer architecture consisting of the presentation layer, application layer, and data layer. The presentation layer includes the Telegram bot and the web interface, which act as entry points for users to interact with the system. These interfaces send user requests to the backend. The application layer is handled by a FastAPI server, which processes all requests and manages the main logic of the system. It also connects with the AI

"Gear Genix" College equipment booking bot



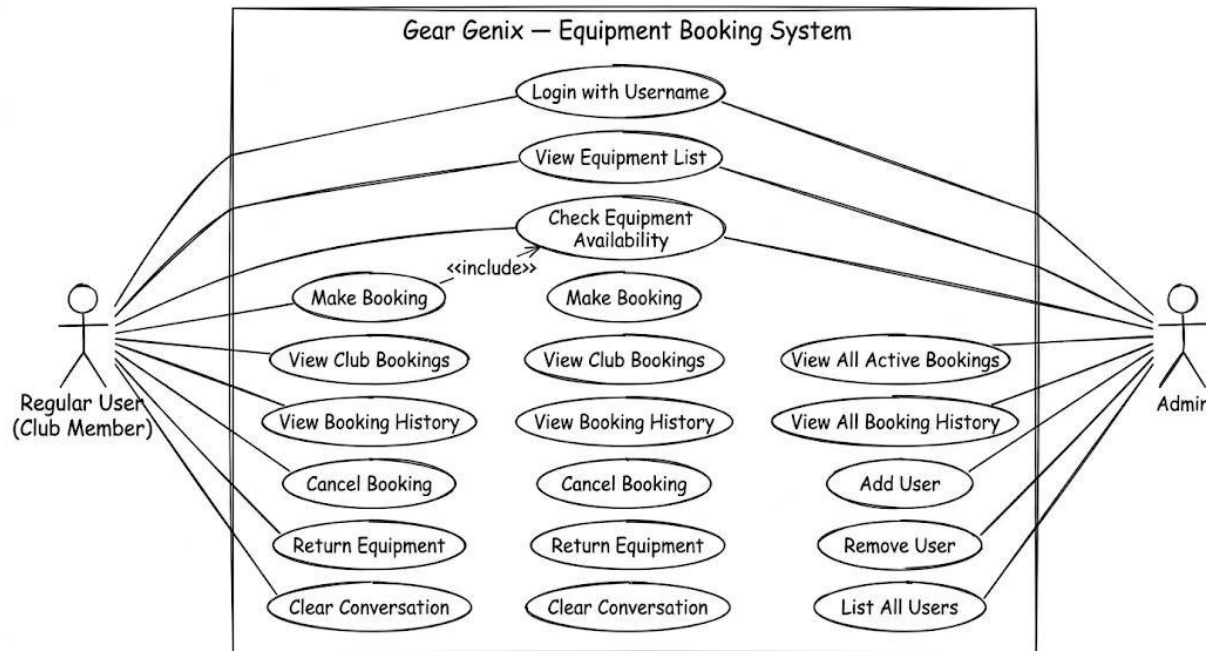
module, which understands the user's message and extracts important details like equipment name and time. The data layer consists of a PostgreSQL database that stores all information related to users, equipment, and bookings. The flow is simple: the user sends a request, the backend processes it with the help of AI, the database is checked or updated, and a response is sent back to the user.

3.3.2 Use Case Diagram

The use case diagram shows how different users interact with the system. There are two main actors: the student (or user) and the admin. Students use the system to perform actions like viewing available equipment, checking availability, booking items, cancelling bookings, and returning equipment after use.

The admin has additional responsibilities such as viewing all bookings and managing equipment records. One important point is that booking equipment always includes checking

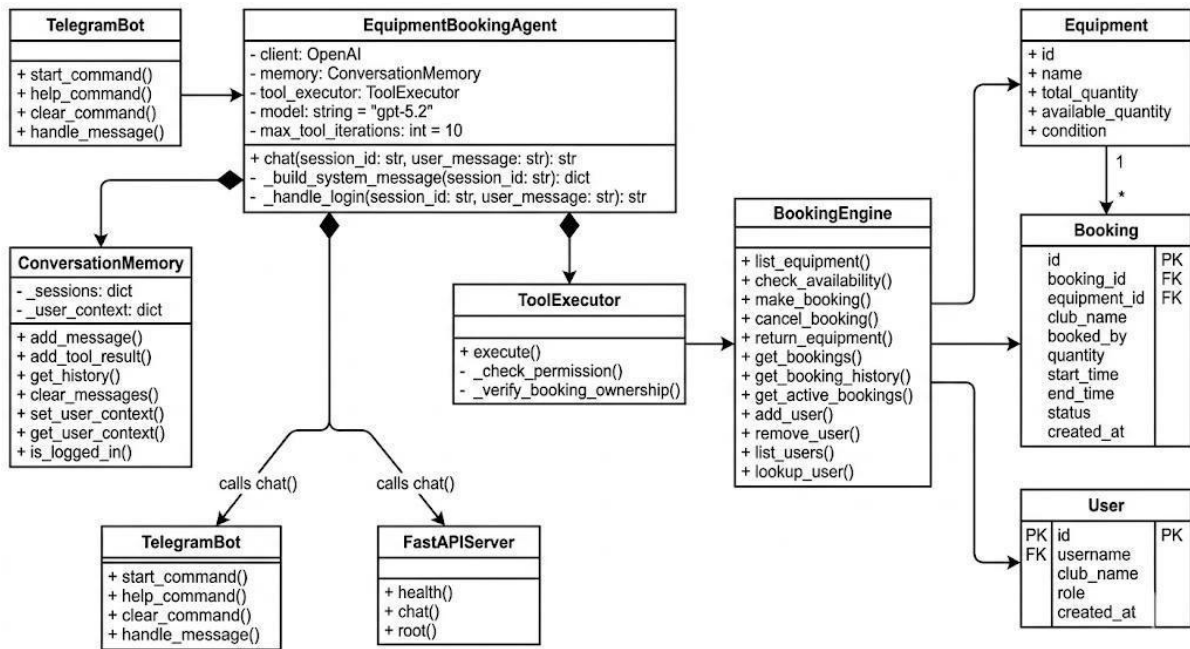
availability first, so the system ensures that no conflicts occur. This diagram helps in understanding all the main functions provided by GearGenix and how users interact with them.



3.3.3 Class Diagram

Figure 3.3 presents the Class Diagram for the GearGenix system. The five primary classes are:

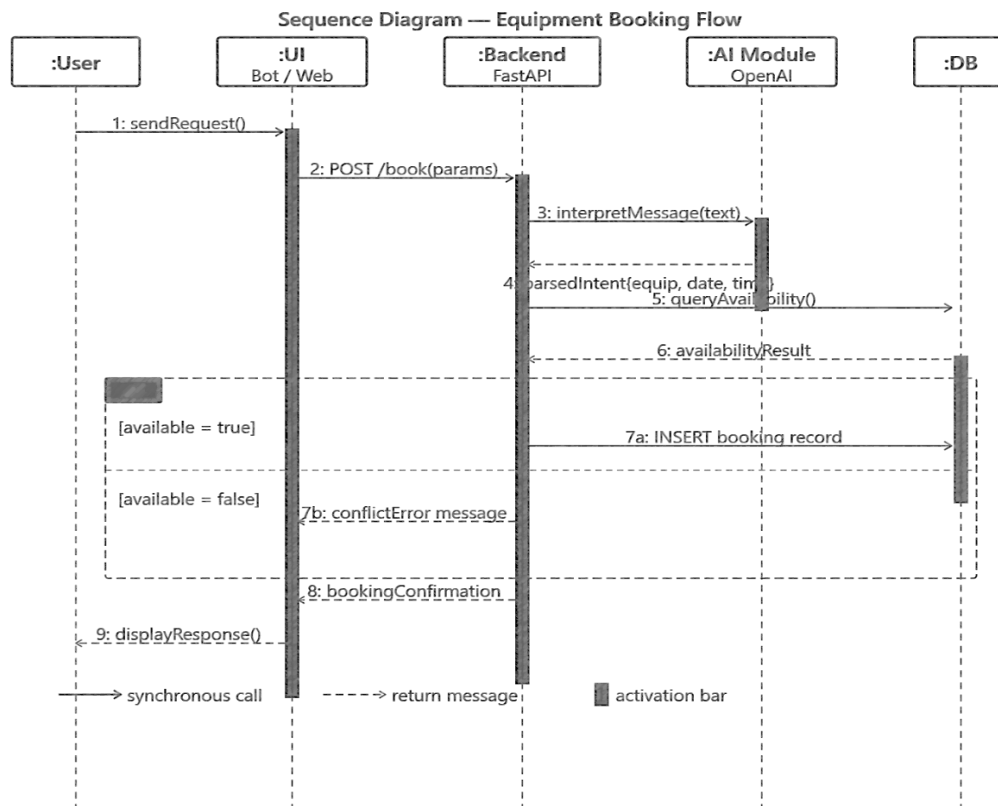
- **User:** id (PK), username, club_name, role (user/admin), created_at
- **Equipment:** id (PK), name, total_quantity, available_quantity, condition
- **Booking:** id (PK), booking_id, equipment_id (FK), club_name, booked_by, quantity, start_time, end_time, status (active/cancelled/returned), created_at
- **EquipmentBookingAgent:** orchestrates the ReAct loop, holds ConversationMemory and ToolExecutor, and dispatches to BookingEngine
- **BookingEngine:** contains all business logic functions (list_equipment, check_availability, make_booking, cancel_booking, return_equipment, get_bookings, get_booking_history, get_active_bookings, add_user, remove_user, list_users, lookup_user)



3.3.4 Sequence Diagram

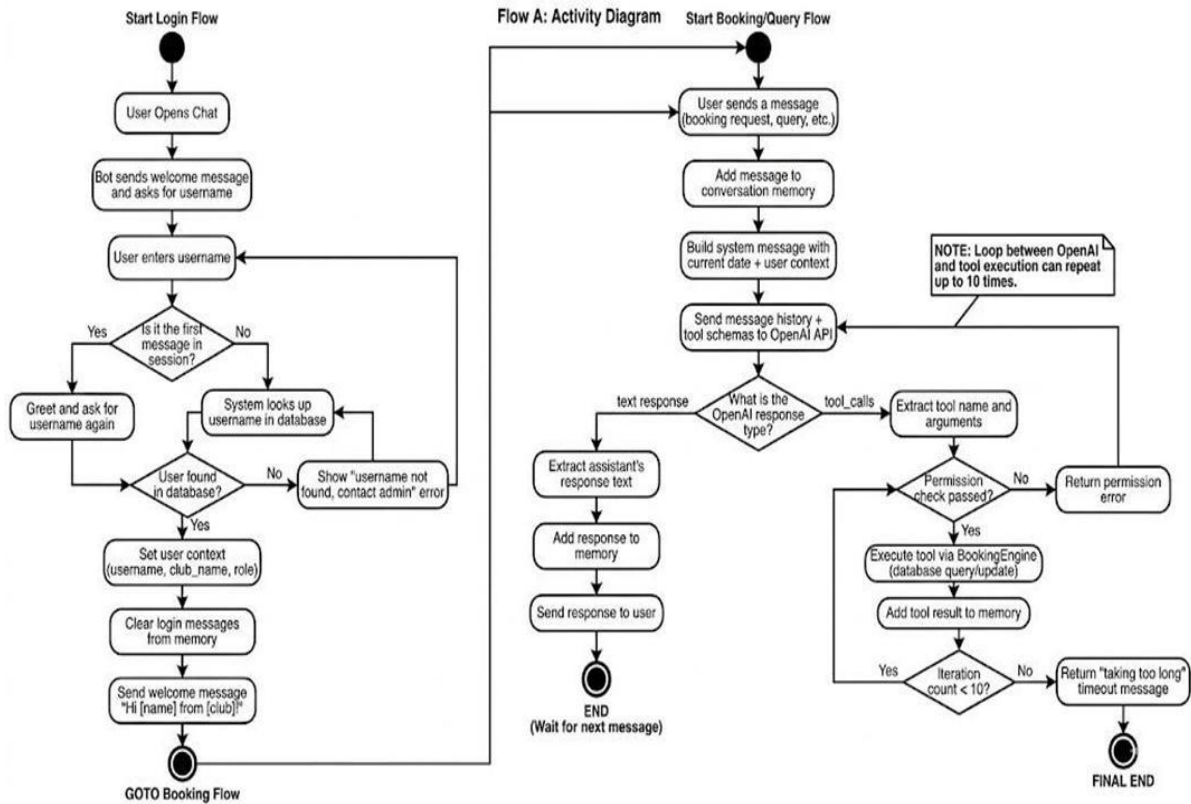
Figure 3.4 presents the Sequence Diagram for the equipment booking flow. The participating lifelines are: User, UI (Bot/Web), Backend (FastAPI), AI Module (OpenAI), and DB (PostgreSQL). The interaction sequence proceeds as follows:

1. The User sends a booking request message to the UI (Telegram bot or web interface).
2. The UI forwards the request as an HTTP POST to the /book endpoint on the FastAPI Backend.
3. The Backend sends the message text to the AI Module for intent interpretation (interpretMessage(text)).
4. The AI Module returns a parsed intent object containing the extracted equipment name, date, start time, and end time (parsedIntent(equip, date, time)).
5. The Backend calls queryAvailability() on the DB, passing the extracted parameters.
6. The DB returns an availabilityResult (true or false).
7. If available = true: The Backend issues an INSERT booking record command to the DB and receives a booking confirmation.
8. If available = false: The Backend generates a conflictError message. The Backend sends either a bookingConfirmation or conflictError response to the UI. The UI calls displayResponse() to present the result to the User.



3.3.5 Activity Diagram

Figure 3.5 presents the Activity Diagram for the Book Equipment workflow. The process is organized across four swim lanes: User, UI/Bot, Backend + AI, and Database. The workflow begins when the user opens the bot or web application and types a booking request. The UI receives and forwards the request to the backend, where the AI module parses the intent. The system queries the database for equipment availability. If the item is available (decision node: Available? = Yes), a booking record is created in the database and a confirmation response is sent back to the UI for display to the user. If the item is not available (Available? = No), a conflict/rejection message is generated instead. The activity diagram ends when the user views the confirmation or rejection result.

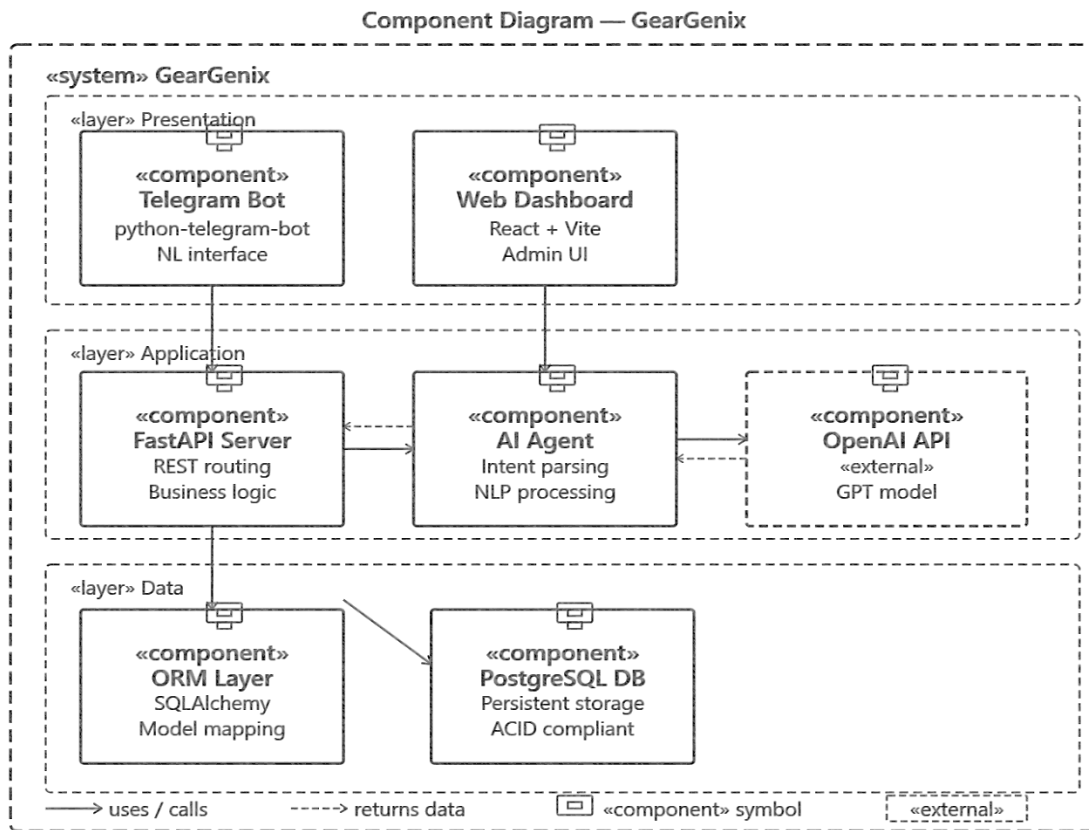


3.3.6 Component Diagram

The component diagram shows the major parts of the GearGenix system and how they are connected. It is divided into three layers: presentation, application, and data.

The presentation layer includes the Telegram bot and the web dashboard, which allow users to interact with the system. These components send user requests to the backend. The application layer contains the FastAPI server, which handles all business logic, and the AI agent, which processes user messages and extracts useful information. The system also connects to the OpenAI API for natural language understanding.

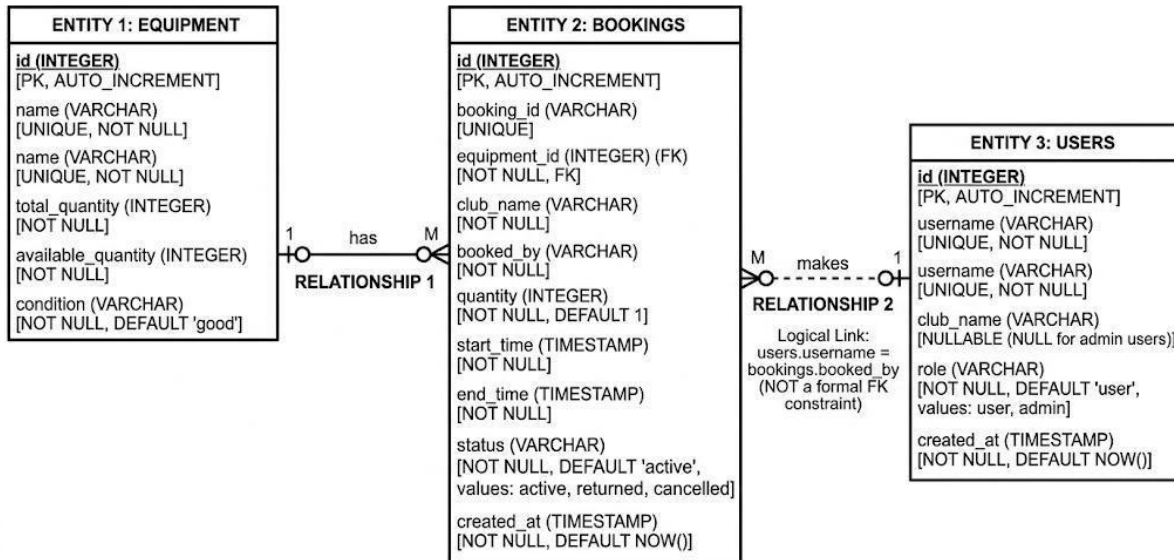
The data layer includes the ORM layer and the PostgreSQL database, which store and manage all system data such as users, equipment, and bookings. Overall, this diagram shows how different components work together to process requests and provide responses efficiently.



3.3.7 ER Diagram

- Figure 3.7 presents the ER Diagram for GearGenix. The three primary entities are **Equipment**, **Bookings**, and **Users**.
- Equipment** → **has** → **Bookings (1:N)**: each equipment item can appear in many bookings; each booking references exactly one equipment item via `equipment_id` (FK).
- Users** → **makes** → **Bookings (1:N, logical link)**: the `bookings.booked_by` column stores the username and is logically linked to `users.username` (not enforced as a formal FK). Equipment stores `id`, `name`, `total_quantity`, `available_quantity`, `condition`. Bookings stores `id`, `booking_id`, `equipment_id`, `club_name`, `booked_by`, `quantity`, `start_time`, `end_time`, `status`, `created_at`. Users stores `id`, `username`, `club_name`, `role`, `created_at`.
- Delete all references to `Booking_Item`, `Admin`, and `Conflict_Log` entities.

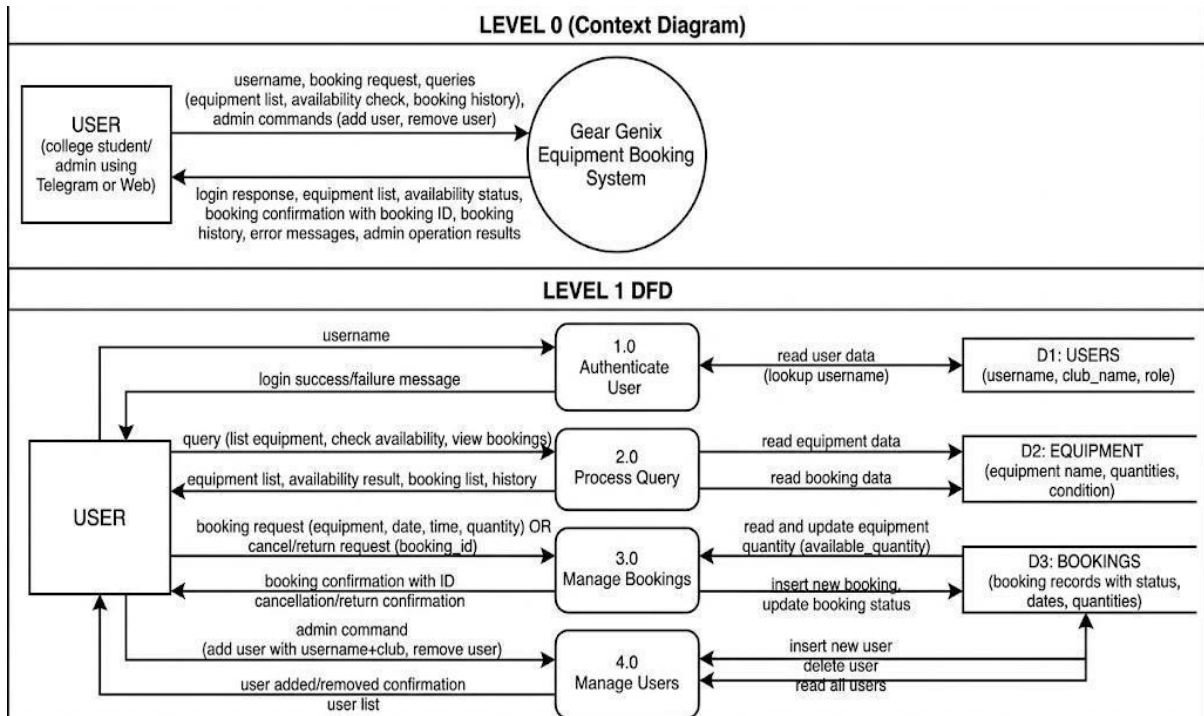
GEAR GENIX EQUIPMENT BOOKING SYSTEM ER DIAGRAM



3.3.8 Data Flow Diagrams (Level 0 and Level 1)

Figure 3.8 in the appendix illustrates the DFD for the GearGenix system. At Level 0 (Context Diagram), the system interacts with two external entities: the Student/User and the Admin/Faculty. The student sends booking requests and receives confirmation or status responses. The admin manages equipment inventory and receives booking reports.

At Level 1 (Process Decomposition), the system is decomposed into six primary processes: (1.0) Authenticate User, (2.0) Parse Natural Language Request, (3.0) Check Availability, (4.0) Process Booking, (5.0) Detect Conflicts, and (6.0) Send Notification. Four data stores support these processes: D1 User Store, D2 Booking Store, D3 Equipment Store, and D4 Conflict Log. The data flow begins with user credentials being authenticated, after which the raw text request is parsed by the AI module, availability is checked against the Equipment Store, and the booking is either committed to the Booking Store or rejected via conflict detection.



3.4 Database Design

3.4.1 Logical Database Design

The logical database design for GearGenix consists of five primary tables organized around the core entities of the system:

Table Name	Primary Key	Foreign Keys	Key Attributes
users	id (INT)	—	username, club_name (nullable for admin), role (user/admin), created_at
equipment	id (INT)	—	name, total_quantity, available_quantity, condition (default 'good')
bookings	id (INT)	equipment_id → equipment	booking_id (UNIQUE), club_name, booked_by, quantity, start_time, end_time, status (active/cancelled/returned), created_at

3.4.2 Physical Database Design

The physical database is implemented in PostgreSQL, accessed through the SQLAlchemy ORM with the following implementation details:

- All primary keys are auto-incrementing INTEGER sequences.

- Role values (user, admin), booking status values (active, cancelled, returned), and equipment condition are stored as VARCHAR with default values, not as native PostgreSQL ENUM types.
- All datetime fields use TIMESTAMP WITHOUT TIME ZONE; time-zone handling is performed at the application layer (IST).
- Foreign key from bookings.equipment_id to equipment.id enforces referential integrity.
- The conflict-detection query and booking insert run within a single SQLAlchemy session transaction.

Chapter 4: Implementation, Testing, and Maintenance

4.1 Introduction to Languages, IDEs, Tools and Technologies

GearGenix is implemented using a modern, production-grade technology stack selected for its performance, scalability, and community support. The following table summarizes the complete technology stack:

Category	Technology	Purpose
Backend Language	Python 3.11	Primary server-side language
Backend Framework	FastAPI	RESTful API server, request routing, business logic
ASGI Server	Uvicorn	Runs the FastAPI application
AI / LLM	OpenAI API (GPT-5.2)	Natural language understanding, ReAct agent, function calling
Database	PostgreSQL 13+	Relational data storage (ACID-compliant)
ORM / Migrations	SQLAlchemy + Alembic	Object-relational mapping and schema migrations
Frontend	Static HTML + CSS + Vanilla JavaScript	Single-page chat UI served by FastAPI from ui/index.html (no build step)
Telegram Interface	python-telegram-bot	Telegram bot for conversational AI interactions
API Documentation	Swagger UI (built-in FastAPI)	Interactive API documentation and testing
Version Control	Git / GitHub	Source code management and collaboration
IDE	Visual Studio Code	Primary development environment
Containerization	Docker (optional)	Deployment containerization for portability
Environment Manager	pip + venv	Python dependency management

Table 4.1: Technology Stack Summary

FastAPI was selected for its asynchronous request handling, automatic OpenAPI documentation generation, and native support for Pydantic data validation. PostgreSQL was chosen for its proven reliability, support for complex queries, and ACID transaction guarantees. The OpenAI API provides the language model backbone for the ReAct agent, enabling function calling with structured JSON parameter extraction. The web UI is a lightweight

single-page chat interface built with plain HTML, CSS, and JavaScript — it is served directly by FastAPI from the `ui/` directory and requires no separate frontend build pipeline.

4.2 Testing Techniques and Test Plans

The testing strategy for GearGenix covers unit testing, integration testing, and user acceptance testing (UAT), aligned with the functional requirements in Chapter 2.

Unit Testing: Individual functions in the conflict detection and booking management modules are tested in isolation using `pytest`. Test cases cover interval-overlap edge cases — partial overlaps, full containment, and adjacent bookings.

Integration Testing: End-to-end API tests verify that FastAPI endpoints correctly coordinate the AI agent and the PostgreSQL database, covering the full workflow from natural-language input to database record creation.

Test ID	Test Case	Input	Expected Output	Result
TC-01	Successful Booking	"Book projector tomorrow 2–4 PM for Robotics Club"	Booking created; confirmation with ID (e.g., B001) returned	Pass
TC-02	Conflict Detection	"Book projector tomorrow 2–4 PM" (slot already booked)	Conflict error returned; no booking created	Pass
TC-03	Booking Cancellation	"Cancel booking B001"	Status set to cancelled; units returned to inventory	Pass
TC-04	List Equipment	"Show available equipment"	Equipment list with total quantity and slot-dependent availability note	Pass
TC-05	Equipment Return	"Return equipment for booking B002"	Status set to returned; units returned to inventory	Pass
TC-06	Admin View Bookings	"Show all active bookings" (admin user)	Full active-booking list across all clubs returned	Pass
TC-07	Adjacent Booking	2–4 PM followed by 4–6 PM request	No conflict; second booking created	Pass

Table 4.2: Test Cases — Equipment Booking Module

4.3 Installation Instructions

Prerequisites:

1. Python 3.11 or higher installed on the server.
2. PostgreSQL 13 or higher installed and running.
3. A valid OpenAI API key.
4. A registered Telegram Bot token (obtained from @BotFather).

Setup

1. Clone the repository: `git clone <repository-url> equipment-booking-bot`
2. Navigate into the project: `cd equipment-booking-bot`
3. Create and activate a virtual environment:
4. Windows: `python -m venv venv && venv\Scripts\activate`
5. macOS/Linux: `python -m venv venv && source venv/bin/activate`
6. Install dependencies: `pip install -r requirements.txt`
7. Configure environment variables in a `.env` file at the project root:
8. `DATABASE_URL` (PostgreSQL connection string) — required
9. `OPENAI_API_KEY` — required
10. `ADMIN_USERNAME` — required (the bootstrap admin's Telegram username)
11. `TELEGRAM_BOT_TOKEN` — optional (omit to run web UI only)
12. Start the application: `python main.py`
13. On startup, `main.py` automatically initialises the database schema, seeds equipment, and seeds the admin user — no separate `init_db.py` step is needed.
14. FastAPI runs on port 8000 in a background thread; the Telegram bot runs in the main thread (skipped if `TELEGRAM_BOT_TOKEN` is not set).

4.4 End User Instructions

Using the Telegram Bot

1. Open Telegram and search for your deployed bot by the handle you registered with BotFather (the `TELEGRAM_BOT_TOKEN` value in `.env` ties the running app to that bot).
2. Send `/start` to receive the welcome message. Other built-in commands:
`/help` — shows example queries.
`/clear` — resets your conversation history with the bot.
3. Type natural-language requests in plain English. Examples:
"What equipment do you have?"
"Is the projector free tomorrow 3–5 PM?"
"Book the projector for Tech Club on 5 May 10 AM to 12 PM."
"Show bookings for Drama Club."
"Cancel booking B007."
"Return equipment for booking B005."
"Who has the projector right now?"
4. Access is gated by Telegram username: only users present in the database (the `ADMIN_USERNAME` from `.env`, plus anyone an admin has added via "add user") can issue booking actions. The bot will reply with confirmation, rejection, or a formatted list as appropriate.

Using the Web Interface

1. Open a browser and navigate to `http://localhost:8000/ui/index.html` (the root path `/` is not served — the UI lives under `/ui/`).
2. The interface is a single chat page powered by the same agent as the Telegram bot. There are no separate "Equipment" or "Bookings" tabs and no separate Admin Panel — every action (listing, booking, cancelling, returning, viewing history, managing users) happens through chat. Quick-action buttons at the top prefill common requests:
List Equipment
Book
My Bookings
Return
3. Admin operations (adding/removing users, listing all users, viewing all-club bookings) are issued as natural-language messages too — they only succeed when the session is

identified as an admin user. Identify yourself first if the bot asks who you are; the agent will reject privileged actions for non-admin sessions.

Chapter 5: Results and Discussions

5.1 User Interface Representation

GearGenix provides two primary user interfaces: a Telegram Bot interface for conversational natural language interactions, and a static web chat interface for browser-based access. Both interfaces route all user requests through the unified /chat endpoint on the FastAPI backend, ensuring consistent processing regardless of the chosen interface.

The Telegram Bot interface presents users with a simple conversational experience. Upon sending /start, the bot greets the user and prompts for a command. Subsequent messages such as "Show me available projectors" or "Book the microphone for Coding Club on 10th May 3 to 5 PM" are processed by the AI agent, which responds with structured confirmations or informative conflict messages.

The web interface is a single-page chat that uses the same /chat endpoint as the Telegram bot. Admin operations are issued as natural-language messages and gated by the role field of the logged-in user.

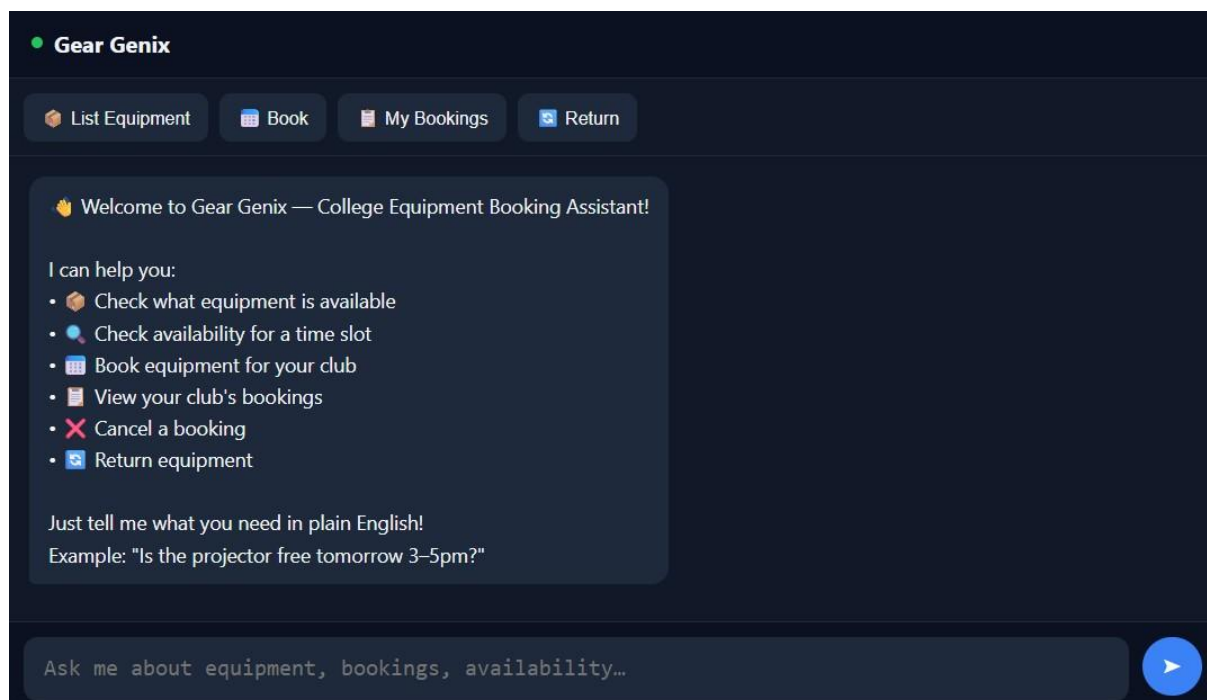


Fig 5.1 web dashboard chat view

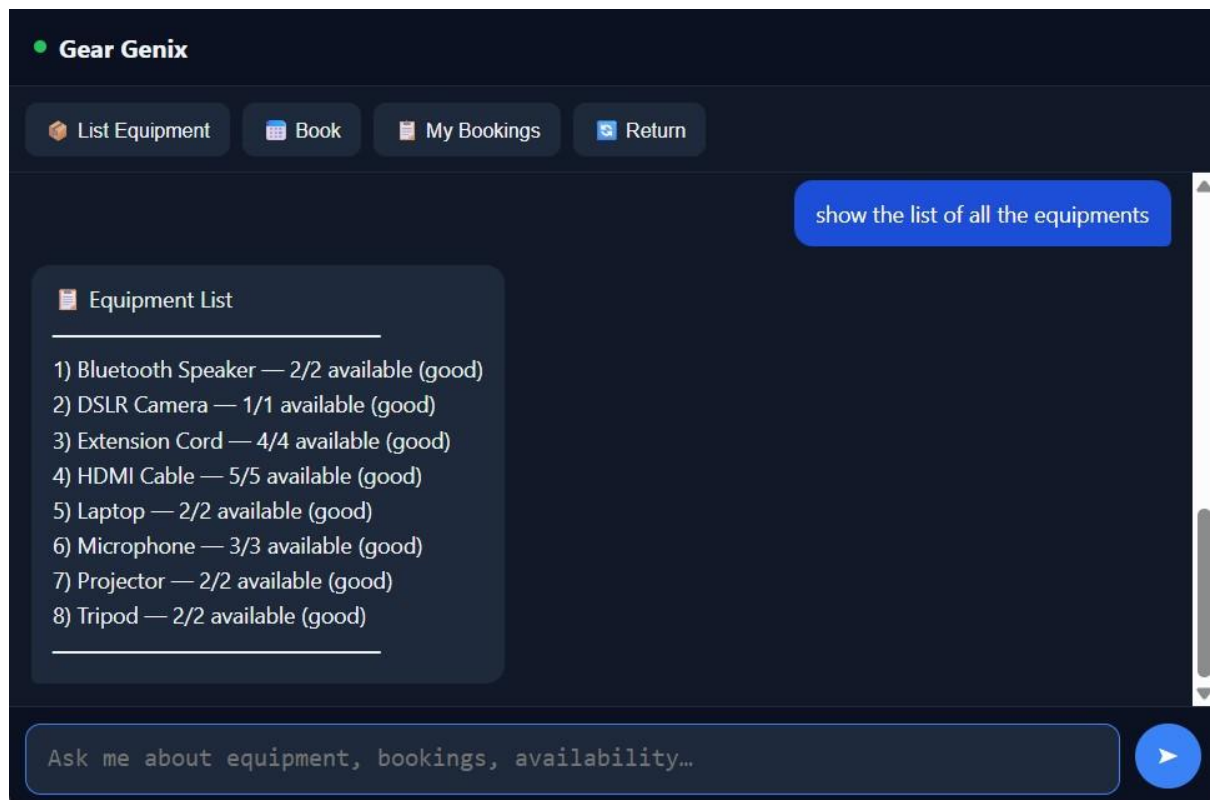


Fig 5.2 equipment listing

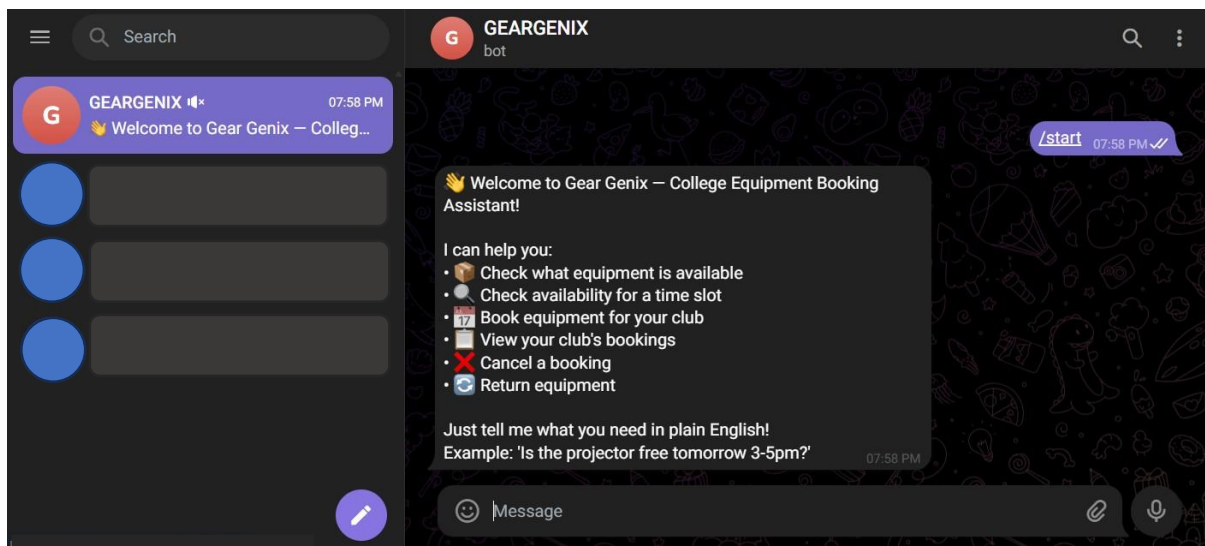


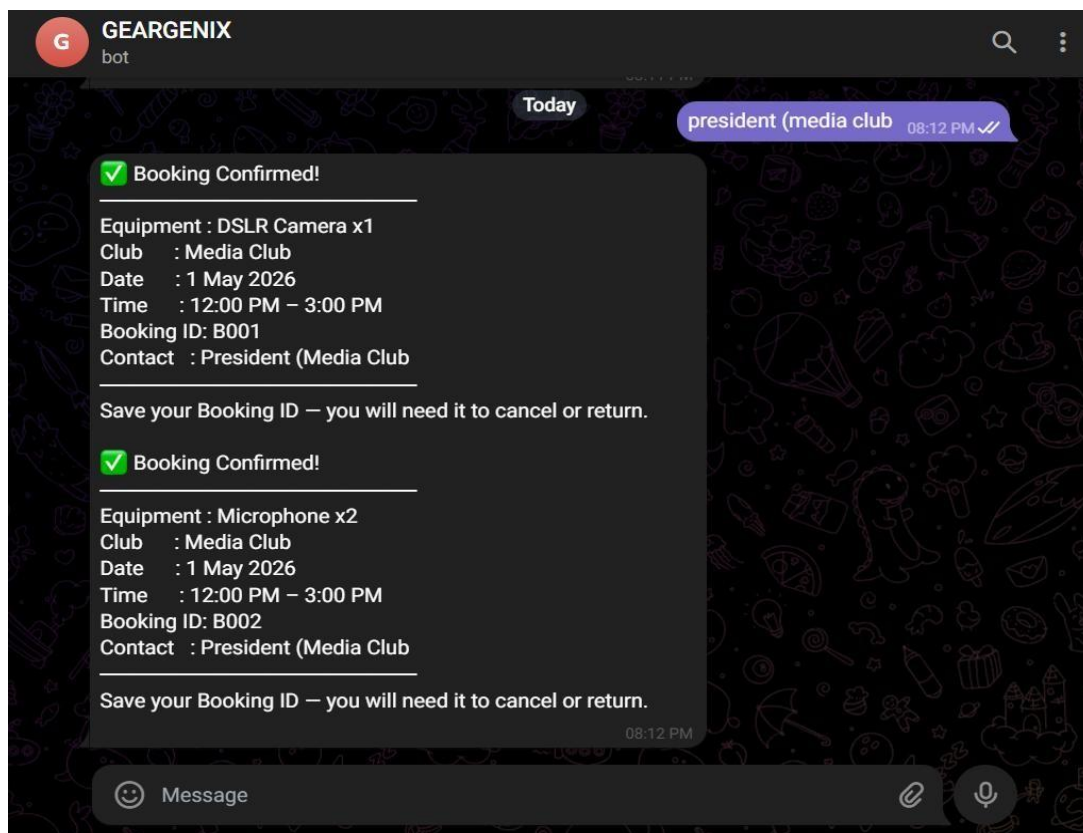
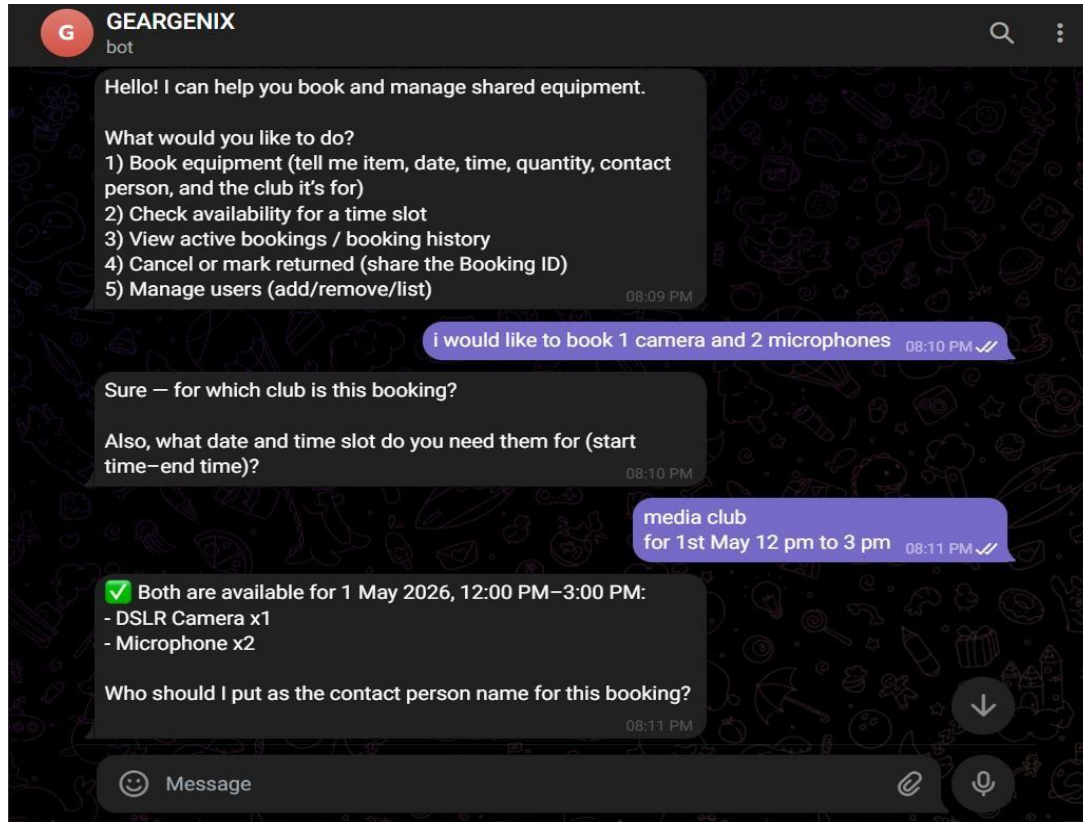
Fig 5.3 Telegram bot interface

5.2 Brief Description of Various Modules

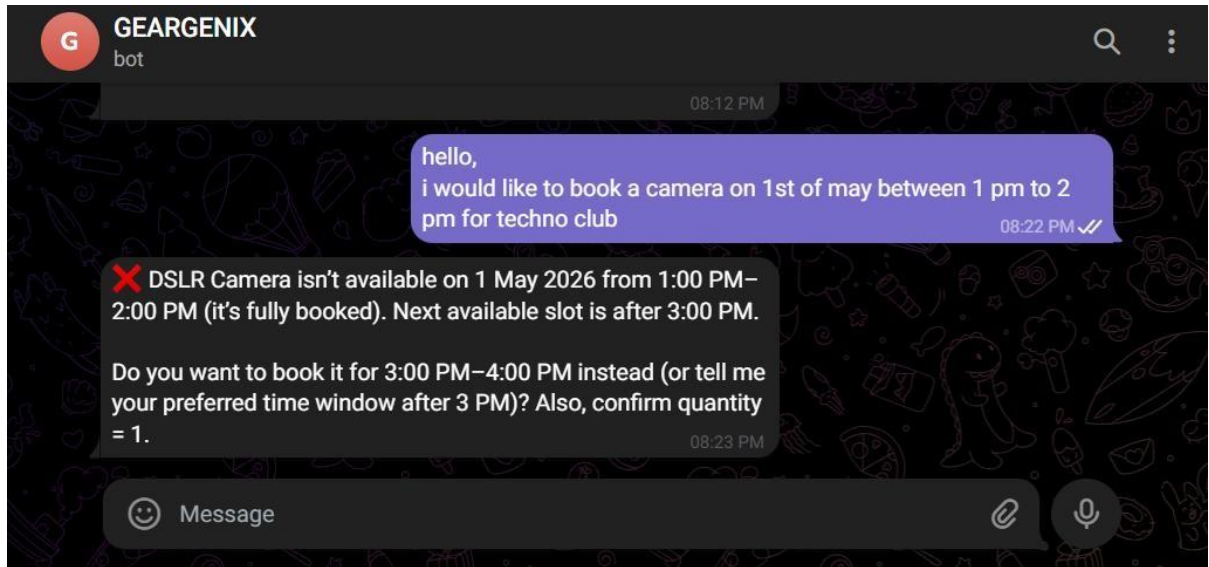
The GearGenix system comprises the following functional modules:

- **AI Agent Module:** The central intelligence module. Implements the ReAct reasoning loop using the OpenAI API. Accepts a user message and conversation history, reasons over the context, selects and calls the appropriate tool function, and generates a natural language response. Supports multi-turn conversations with full context retention.
- **Authentication Module:** Identifies the user by Telegram username (or by username entered on first message in the web UI) and looks them up in the users table to determine their role. There are no passwords, sessions, or JWT tokens — access is gated purely by username presence and role.
- **Equipment Management Module:** Provides CRUD operations for equipment records. Administrators can add new equipment, update descriptions, change status (available/booked/under maintenance), and deactivate retired items. Students can view the equipment listing and availability status.
- **Booking Management Module:** Manages the complete booking lifecycle. On booking creation, the module invokes the Conflict Detection Module, records the booking in the database if no conflict is found, generates a human-readable booking ID, and returns a confirmation. Supports cancellation, status viewing, and return confirmation.
- **Conflict Detection Module:** Implements the interval overlap algorithm at the database query level. Logs all conflict checks, including results, in the Conflict_Log table for audit purposes.
- **Telegram Bot Interface Module:** Built with python-telegram-bot. Receives Telegram messages via long polling, passes them to the AI Agent Module via the /chat endpoint, and returns the agent's response as a Telegram message.
- **Web Interface Module:** A static HTML/CSS/JavaScript page served by FastAPI from ui/index.html. Sends user messages to the /chat endpoint and renders the agent's responses.

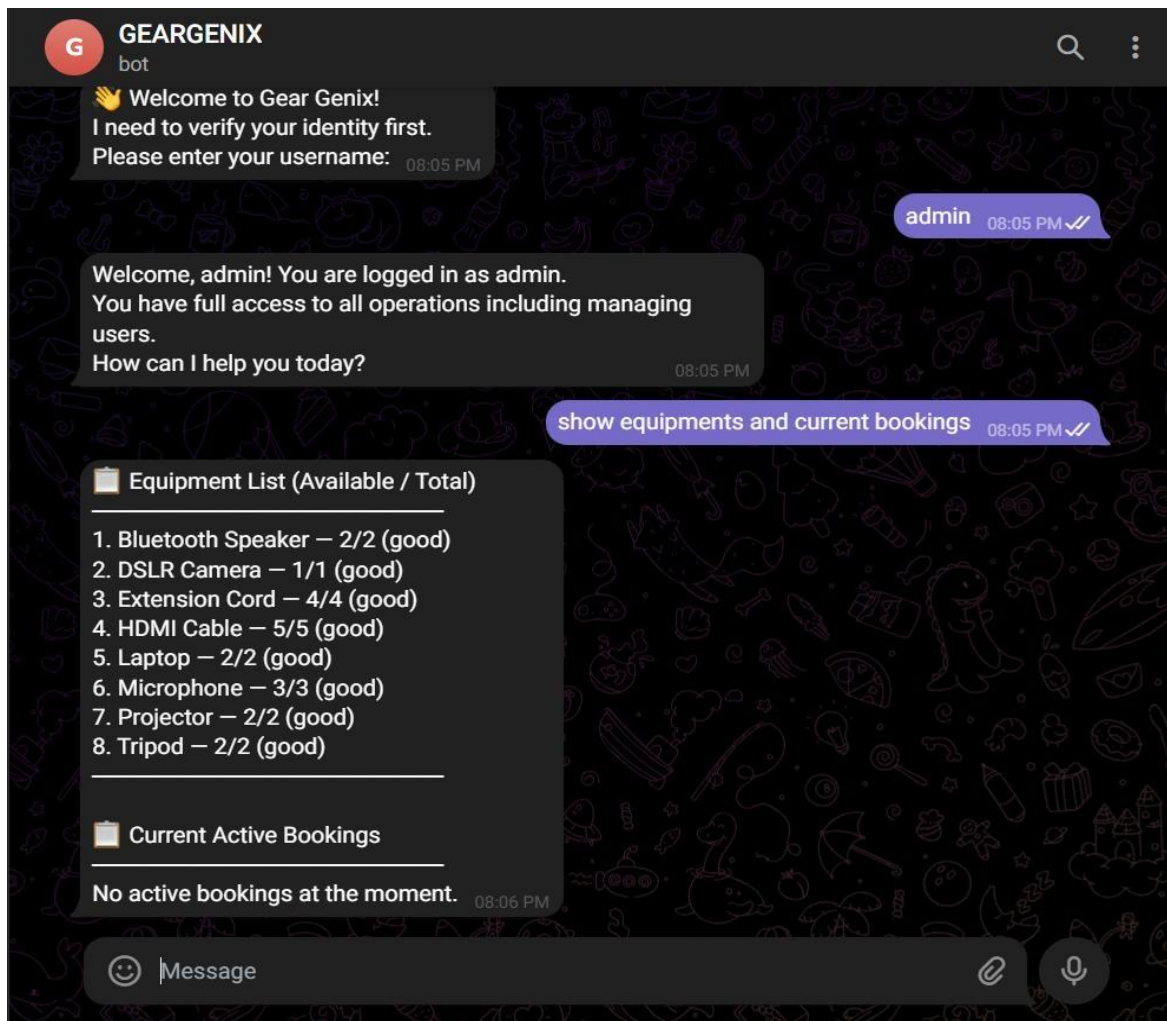
5.3 Snapshots of System



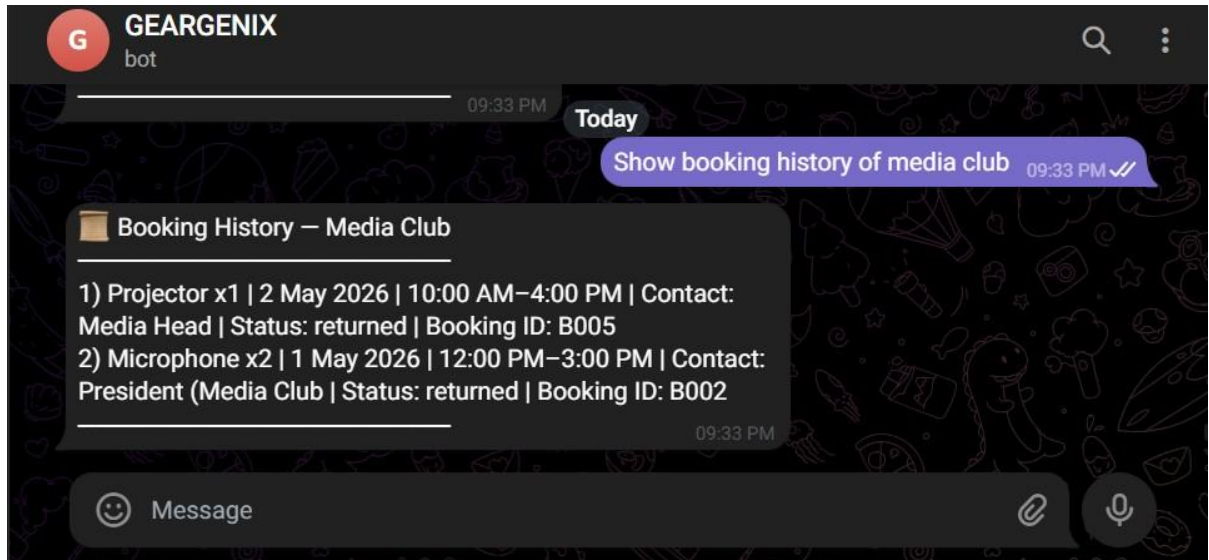
(a) Telegram bot conversation showing a successful booking flow



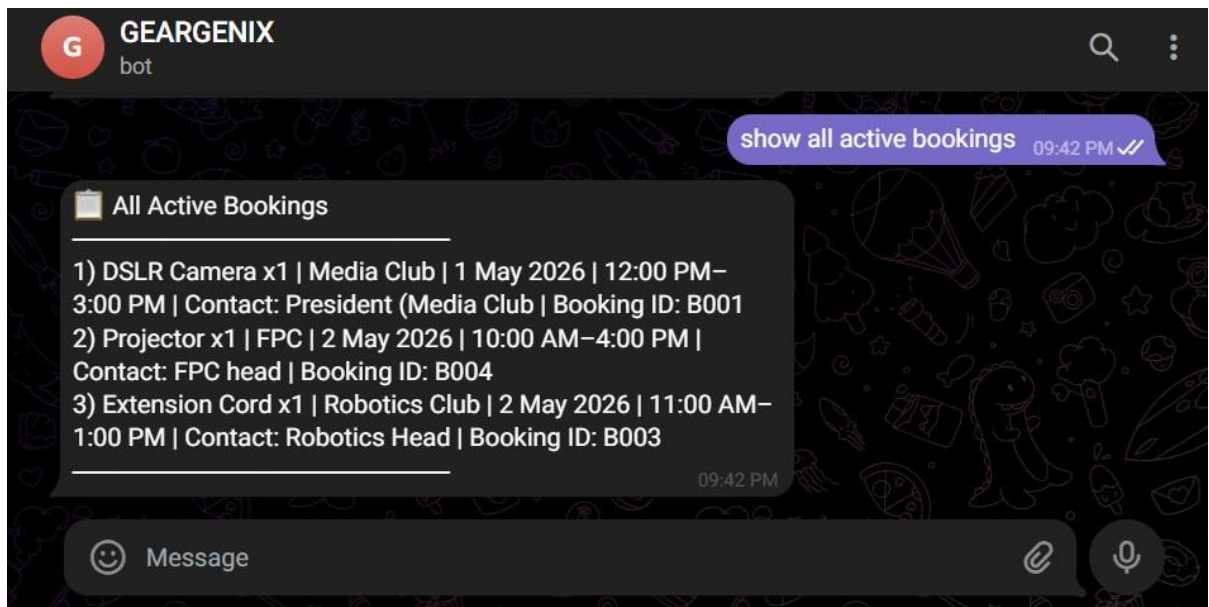
(b) Telegram bot showing a conflict rejection message



(c) Web dashboard — equipment listing view



(d) Booking history for a student



(e) All bookings view

5.4 Back-End Representation

The API layer exposes two RESTful endpoints: GET /health (health check) and POST /chat (the unified AI agent endpoint that handles all user actions including listing, booking, cancelling, and returning equipment via natural language). All operations are routed through /chat; there are no separate REST endpoints for individual booking actions.

The service layer contains the business logic for each endpoint, coordinating between the AI module, the ORM, and the database. The ORM layer uses SQLAlchemy models that map directly to the PostgreSQL tables described in Section 3.4. The database layer executes all queries within transactions, with the conflict detection query using a SELECT FOR UPDATE lock to prevent concurrent conflicting writes.

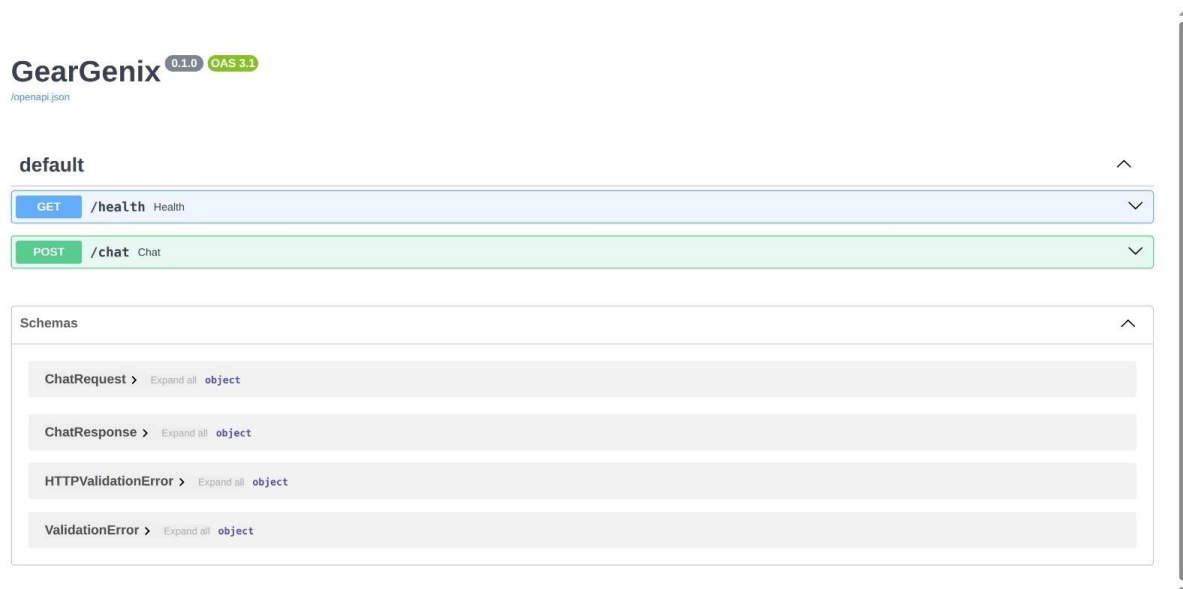


Fig 5.5 FastAPI Swagger UI

bookings

General Columns Advanced Constraints Partitions Parameters Security SQL

Inherited from table(s)

	Name	Data type	Length/Precision	Scale	Not NULL?	Primary key?	Default
	id	integer			☒	☒	nextval('booking
	booking_id	character varying			☐	☐	
	equipment_id	integer			☒	☐	
	club_name	character varying			☒	☐	
	booked_by	character varying			☒	☐	
	quantity	integer			☒	☐	1
	start_time	timestamp without time zone			☒	☐	
	end_time	timestamp without time zone			☒	☐	
	status	character varying			☒	☐	'active'::characte
	created_at	timestamp without time zone			☒	☐	now()

equipment

General Columns Advanced Constraints Partitions Parameters Security SQL

Inherited from table(s)

	Name	Data type	Length/Precision	Scale	Not NULL?	Primary key?	Default
	id	integer			☒	☒	nextval('eqi
	name	character varying			☒	☐	
	total_quantity	integer			☒	☐	
	available_quantity	integer			☒	☐	
	condition	character varying			☒	☐	'good'::char

users

General Columns Advanced Constraints Partitions Parameters Security SQL

Inherited from table(s)

	Name	Data type	Length/Precision	Scale	Not NULL?	Primary key?	Default
	id	integer			☒	☒	nextval('t
	username	character varying			☒	☐	
	club_name	character varying			☐	☐	
	role	character varying			☒	☐	'user'::ch
	created_at	timestamp without			☒	☐	now()

Fig 5.6 screenshot of the pgAdmin database schema view

5.5 Snapshots of Database Tables

The GearGenix PostgreSQL database contains three primary tables: users, equipment, and bookings. Sample records in the booking table include fields such as booking_id, user_id, booking_date, start_time, end_time, status, and notes. The conflict_log table records every call to the conflict detection algorithm, storing the booking_id, the timestamp of the check, and a Boolean flag indicating whether a conflict was detected.

	id [PK] integer	booking_id character varying	equipment_id integer	club_name character varying	booked_by character varying	quantity integer	start_time timestamp without time zone	end_time timestamp without time zone	status character	created_at timestamp without time zone
1	4	B001	1	FPC	Anushka	1	2026-05-01 12:00:00	2026-05-01 14:00:00	active	2026-04-29 17:17:30.23449
2	5	B002	1	FPC	Anushka	2	2026-05-01 15:00:00	2026-05-01 17:00:00	active	2026-04-29 17:20:47.241588

Fig 5.7 Bookings Table

	id [PK] integer	name character varying	total_quantity integer	available_quantity integer	condition character varying
1	1	Projector	2	1	good
2	2	Microphone	3	3	good
3	3	Bluetooth Speaker	2	2	good
4	4	Laptop	2	2	good
5	5	HDMI Cable	5	5	good
6	6	Extension Cord	4	4	good
7	7	DSLR Camera	1	1	good
8	8	Tripod	2	2	good

Fig 5.8 Equipments Table

	id [PK] integer	username character varying	club_name character varying	role character varying	created_at timestamp without time zone
1	1	admin	[null]	admin	2026-04-23 00:05:08.351972
2	7	FPC_head	FPC	user	2026-04-29 17:14:34.808326

Fig 5.9 User Table

Chapter 6: Summary and Conclusions

This report has presented GearGenix, an AI-powered campus equipment management system designed to resolve the chronic resource coordination failures experienced by colleges with large numbers of clubs and shared equipment. The problem was quantified clearly: an eightyfive percent coordination failure rate across thirteen institutions, three to five hours of

daily manual coordination wasted per club, and seventy percent of events delayed due to double bookings, with faculty spending fifteen or more hours per week on equipment queries.

GearGenix addresses this problem through a conversational AI agent built on the OpenAI ReAct pattern with function calling, enabling users to manage equipment bookings through natural language via a Telegram bot or React web dashboard. The system's core innovation is the integration of real-time conflict detection—implemented as an interval overlap algorithm executed within ACID-compliant database transactions—with an AI agent capable of multistep tool chaining, providing a seamless and autonomous booking experience without human intermediary.

The system architecture follows a clean three-layer separation of concerns: the presentation layer (Telegram Bot and Web Dashboard), the application layer (FastAPI Server and OpenAI AI Agent), and the data layer (PostgreSQL via SQLAlchemy ORM). Eight core functional requirements were identified, designed, and implemented, covering the complete equipment lifecycle from listing and availability checking through booking, cancellation, and return. The database design was normalized to Third Normal Form with appropriate indexing to support performant conflict detection under concurrent load.

The implementation demonstrates the practical viability of applying modern AI agent architectures to solve real-world institutional management problems. GearGenix eliminates manual coordination overhead, ensures complete booking accountability, and provides administrators with full visibility into equipment usage across all clubs. The system is designed to scale beyond its initial deployment at Medicaps University, Indore, and represents a replicable solution for educational institutions nationwide.

Chapter 7: Future Scope

While GearGenix in its current form delivers a complete and functional campus equipment management solution, several enhancements are identified for future development iterations:

- **Mobile Application:** Development of dedicated native iOS and Android applications to provide a richer mobile experience beyond the current Telegram bot interface, with push notifications, offline viewing of booking history, and biometric authentication.
- **QR Code Equipment Tracking:** Integration of QR code scanning at the point of physical equipment pickup and return, enabling automated status updates and eliminating the reliance on manual return confirmations through the chat interface.
- **Intelligent Notification Reminders:** Automated proactive reminders sent to users before their booking start time, escalating alerts for overdue equipment returns, and admin notifications for equipment that has been in use beyond its scheduled return window.

- **Analytics Dashboard:** A comprehensive usage analytics module for administrators, providing insights into equipment utilization rates, most-requested items, peak booking periods, and club-wise consumption patterns to inform procurement decisions.
- **Multi-Institution Deployment:** Extension of the system to support multiple colleges under a single cloud deployment, with institution-level data isolation, shared equipment categories, and cross-institutional booking capability for inter-college events.
- **Offline Mode and Progressive Web App:** Implementation of Progressive Web App (PWA) capabilities to allow users to view their bookings and equipment information even without an active internet connection, with synchronization upon reconnection.
- **Voice Interface Integration:** Integration with voice assistants such as Google Assistant or Amazon Alexa to allow hands-free equipment booking through voice commands, leveraging the existing natural language processing backbone.
- **Predictive Availability:** Machine learning models trained on historical booking data to predict equipment demand during peak periods, enabling administrators to proactively manage equipment availability and prevent bottlenecks before they occur.

Appendix

Appendix A: Sample Bot Conversation

User : /start

Bot : Welcome to Gear Genix! Please enter your username.

User : president_medioclub

Bot : Welcome! How can I help you today?

User : Book 1 camera and 2 microphones for Media Club, 1 May 12 PM–3 PM Bot

: Both available for 1 May 2026, 12:00 PM–3:00 PM.

Who is the contact person?

User : President (Media Club) Bot

: Booking Confirmed!

DSLR Camera x1 | Booking ID: B001

Microphone x2 | Booking ID: B002

Save your Booking ID — needed to cancel or return.

Appendix B: Sample /chat API Payload

Request

```
{ "session_id": "f4a1c9e2-...", "message": "Is the projector free tomorrow 3–5 PM?" }
```

Response

```
{ "response": " Projector is available on 30 April 2026 from 3:00–5:00 PM (2/2 free)." }
```

Appendix C: Sample .env Configuration

```
DATABASE_URL=postgresql://username:password@localhost:5432/geargenix
OPENAI_API_KEY=sk-xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
ADMIN_USERNAME=admin
TELEGRAM_BOT_TOKEN=0000000000:AAAAAAAAAAAAAAAAAAAAAAAAAAAAA
```

References

- [1] OpenAI, "Function Calling Guide," OpenAI Platform Documentation, 2025. [Online]. Available: <https://platform.openai.com/docs/guides/function-calling>. Accessed: 29/04/2026.
- [2] S. Ramírez, "FastAPI Documentation," 2025. [Online]. Available: <https://fastapi.tiangolo.com/>. Accessed: 29/04/2026.
- [3] PostgreSQL Global Development Group, "PostgreSQL 15 Documentation," 2024. [Online]. Available: <https://www.postgresql.org/docs/15/>. Accessed: 29/04/2026.
- [4] M. Bayer, "SQLAlchemy 2.0 Documentation," 2025. [Online]. Available: <https://docs.sqlalchemy.org/>. Accessed: 29/04/2026.
- [5] python-telegram-bot Contributors, "python-telegram-bot Documentation," v21.x, 2025. [Online]. Available: <https://docs.python-telegram-bot.org/>. Accessed: 29/04/2026.
- [6] S. Yao *et al.*, "ReAct: Synergizing Reasoning and Acting in Language Models," in *Proc. ICLR*, 2023.
- [7] OpenAI, "API Reference and Function Calling Guide," <https://platform.openai.com/docs/>. Accessed: 29/04/2026.
- [8] S. Ramírez, "FastAPI Documentation," <https://fastapi.tiangolo.com/>. Accessed: 29/04/2026.
- [9] PostgreSQL Global Development Group, "PostgreSQL Documentation," <https://www.postgresql.org/docs/>. Accessed: 29/04/2026.

- [10] M. Bayer, "SQLAlchemy Documentation," <https://docs.sqlalchemy.org/>. Accessed: 29/04/2026.
- [11] python-telegram-bot Contributors, "python-telegram-bot Documentation," <https://docs.python-telegram-bot.org/>. Accessed: 29/04/2026.

Bibliography

- [B1] J. Wei *et al.*, "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models," in *Proc. NeurIPS*, 2022.
- [B2] R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, Pearson, 7th ed., 2015.
- [B3] I. Sommerville, *Software Engineering*, Pearson, 10th ed., 2015.
- [B4] G. Booch, J. Rumbaugh, and I. Jacobson, *The Unified Modeling Language User Guide*, Addison-Wesley, 2nd ed., 2005.
- [B5] Docker Inc., "Docker Documentation," <https://docs.docker.com/>. Accessed: 29/04/2026.